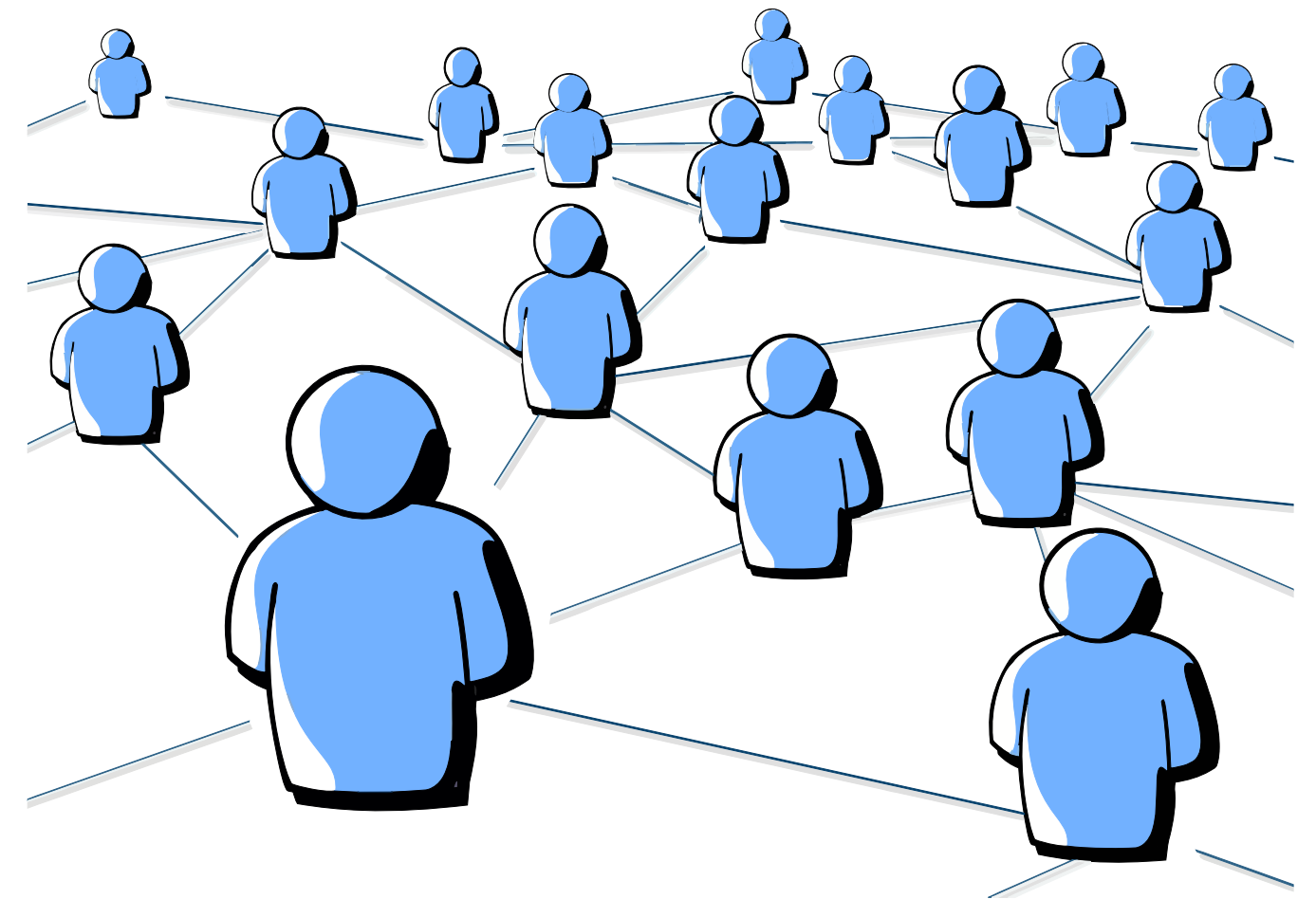


Ski Resort Application

A multi-tier, multithreaded, scalable distributed system

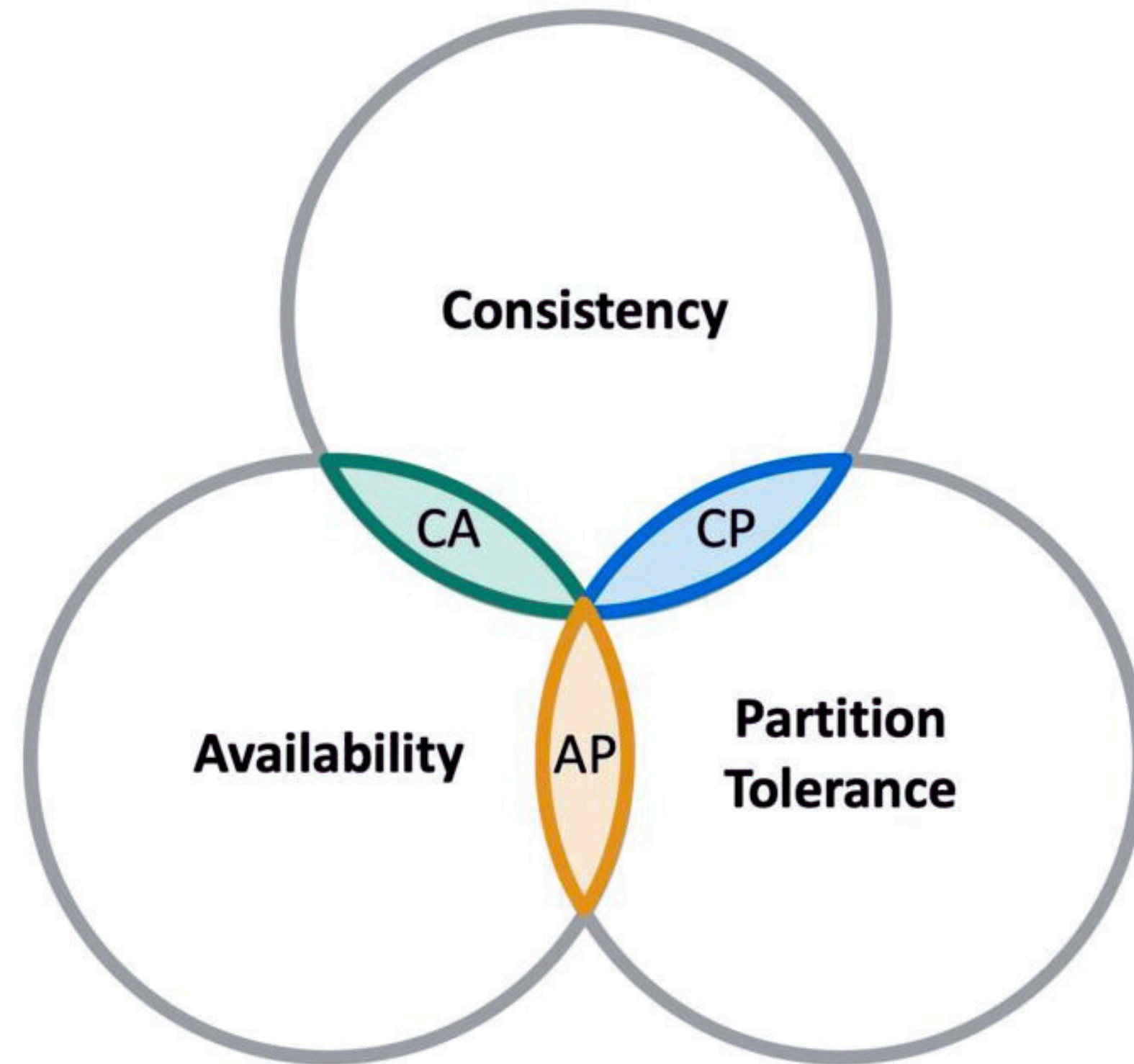


Presented by

Riddhi Gohil

Consistency means that data is the same across the cluster, so you can read or write from/to any node and get the same data.

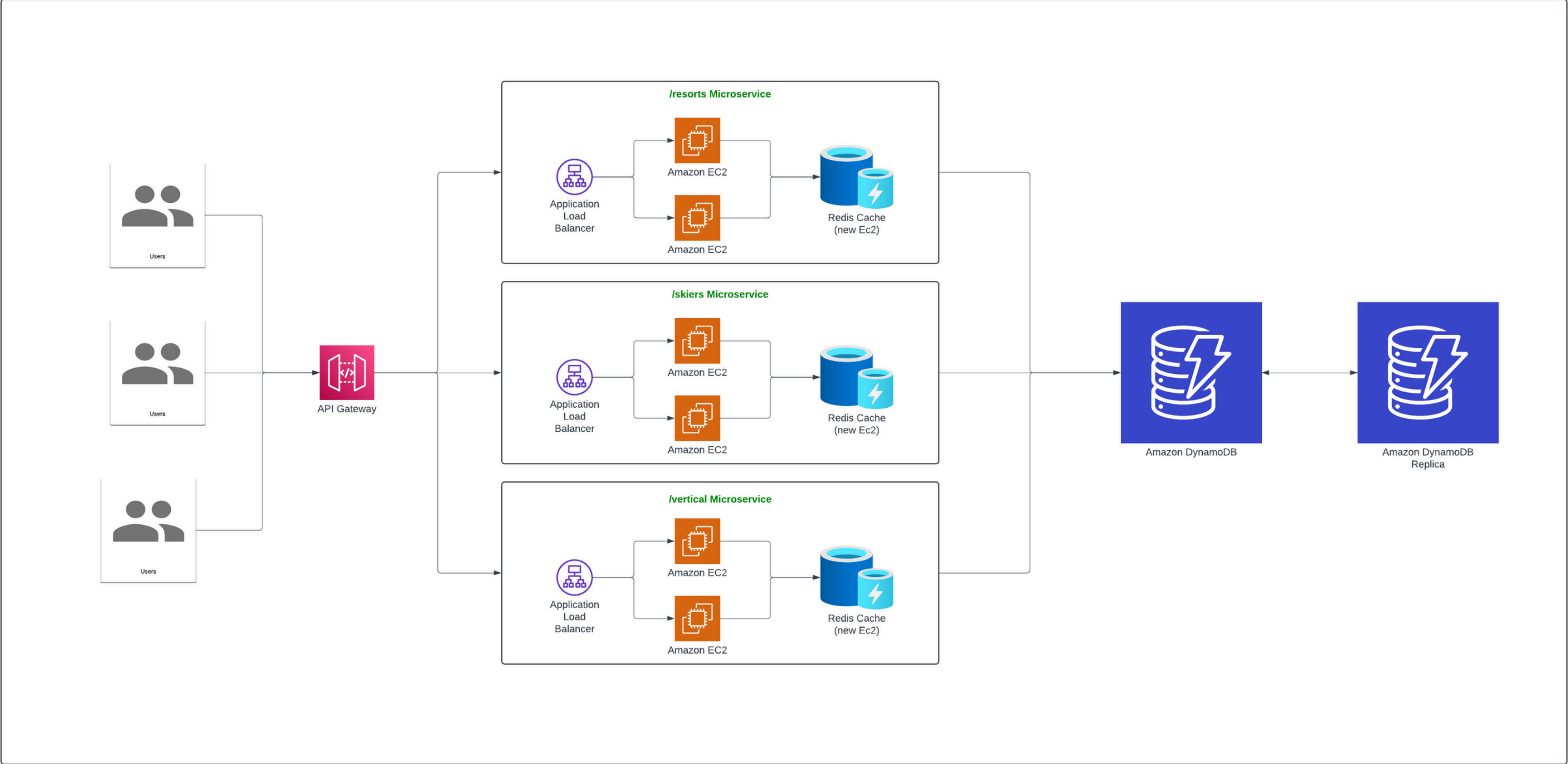
Approach



Availability means the ability to access the cluster even if a node in the cluster goes down.

Partition tolerance means that the cluster continues to function even if there is a partition/communication break between two nodes.

Architecture



Features

01 Microservice Architecture

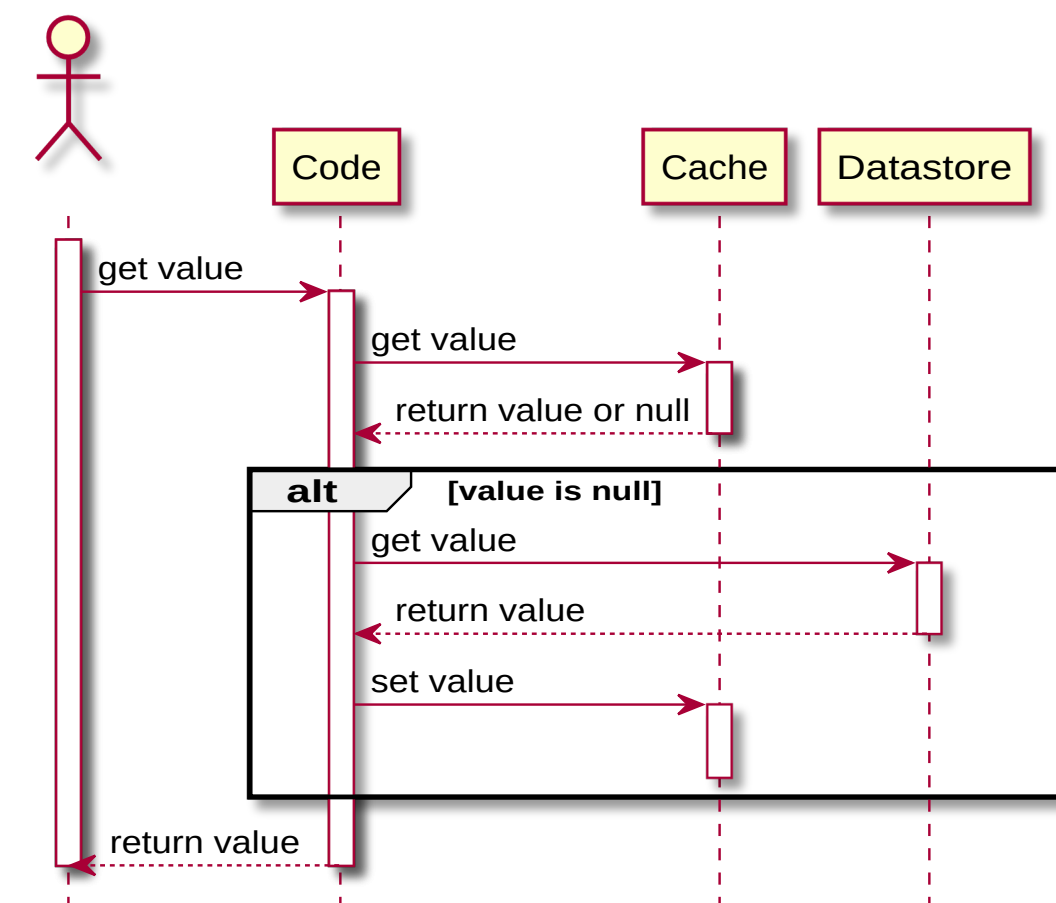
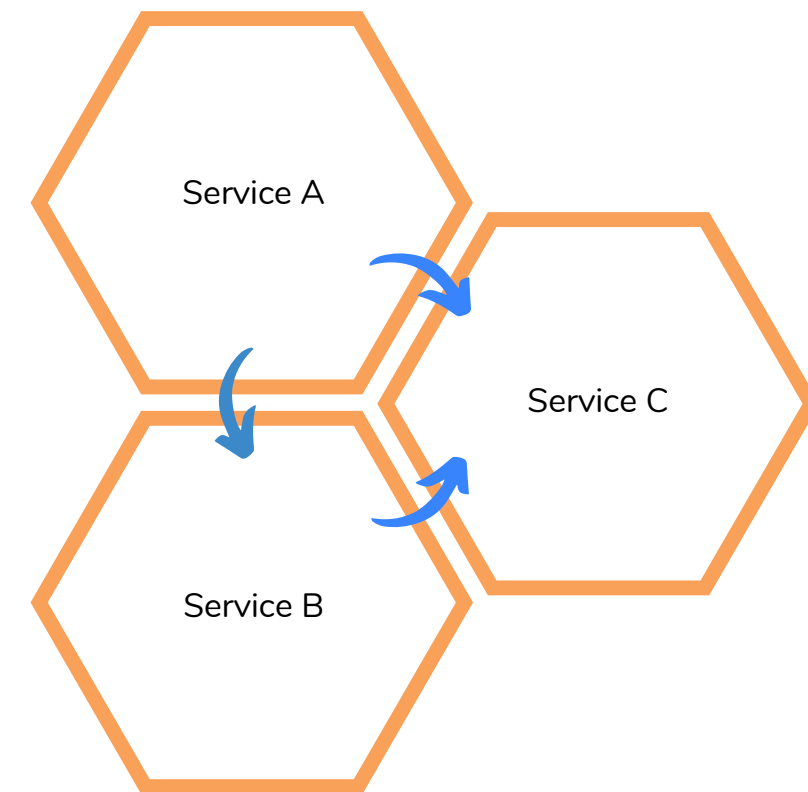
- Decoupled, Modular-Independently Deployable
- Highly Scalable

02 Load Balancing

- Handle fluctuations in demand while maintaining responsiveness and availability.
- Helps the system be available in peak traffic.

03 Read-through caching

- Cache frequently accessed data to reduce load on underlying database.
- Reduce Latency



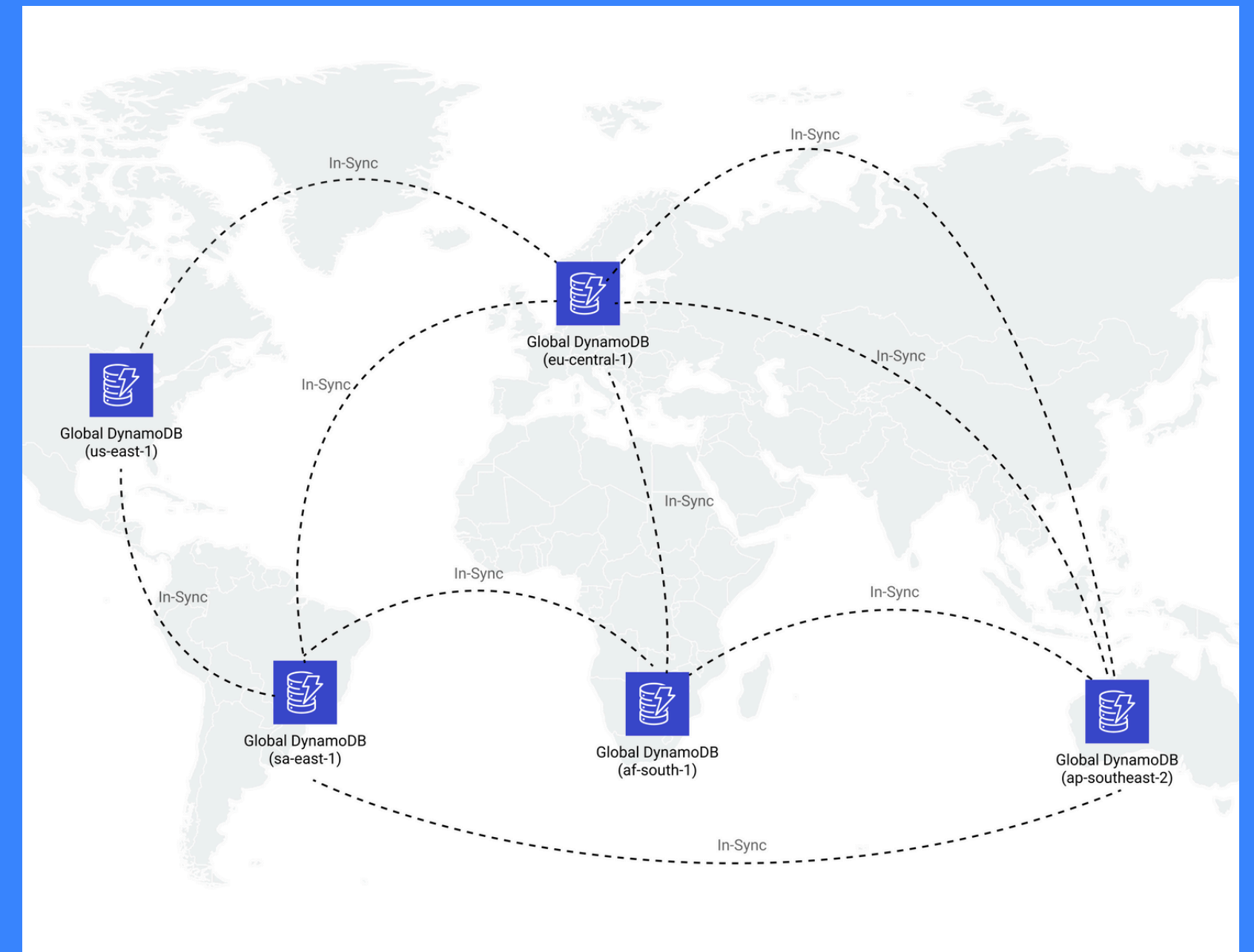
Features

04 API Gateway

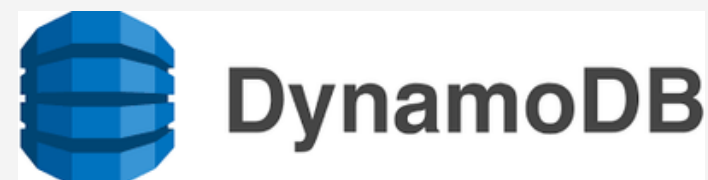
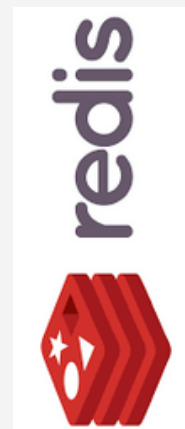
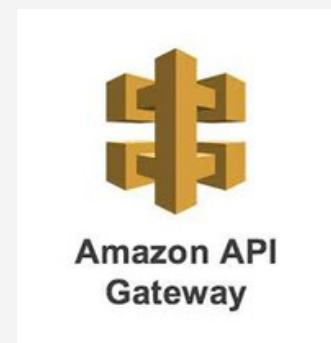
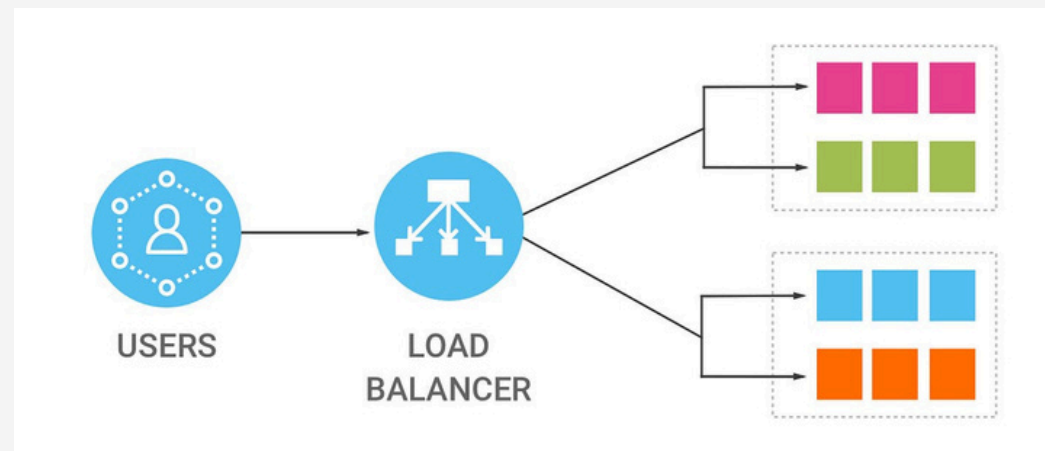
- Simplifies request routing and management
- Common Endpoint

05 Distributed Database

- Data Replication, Distribution and Partition
- Global Secondary Indexes (GSIs)
- Quorum-Based Consistency



Tradeoffs



Complexity

Multiple micro-services, each with its own **cache** and **load balancer**, increases the complexity of the system.



Cost

Load Balancers, Instances, API Gateway, DynamoDB, and **Cache** add additional costs

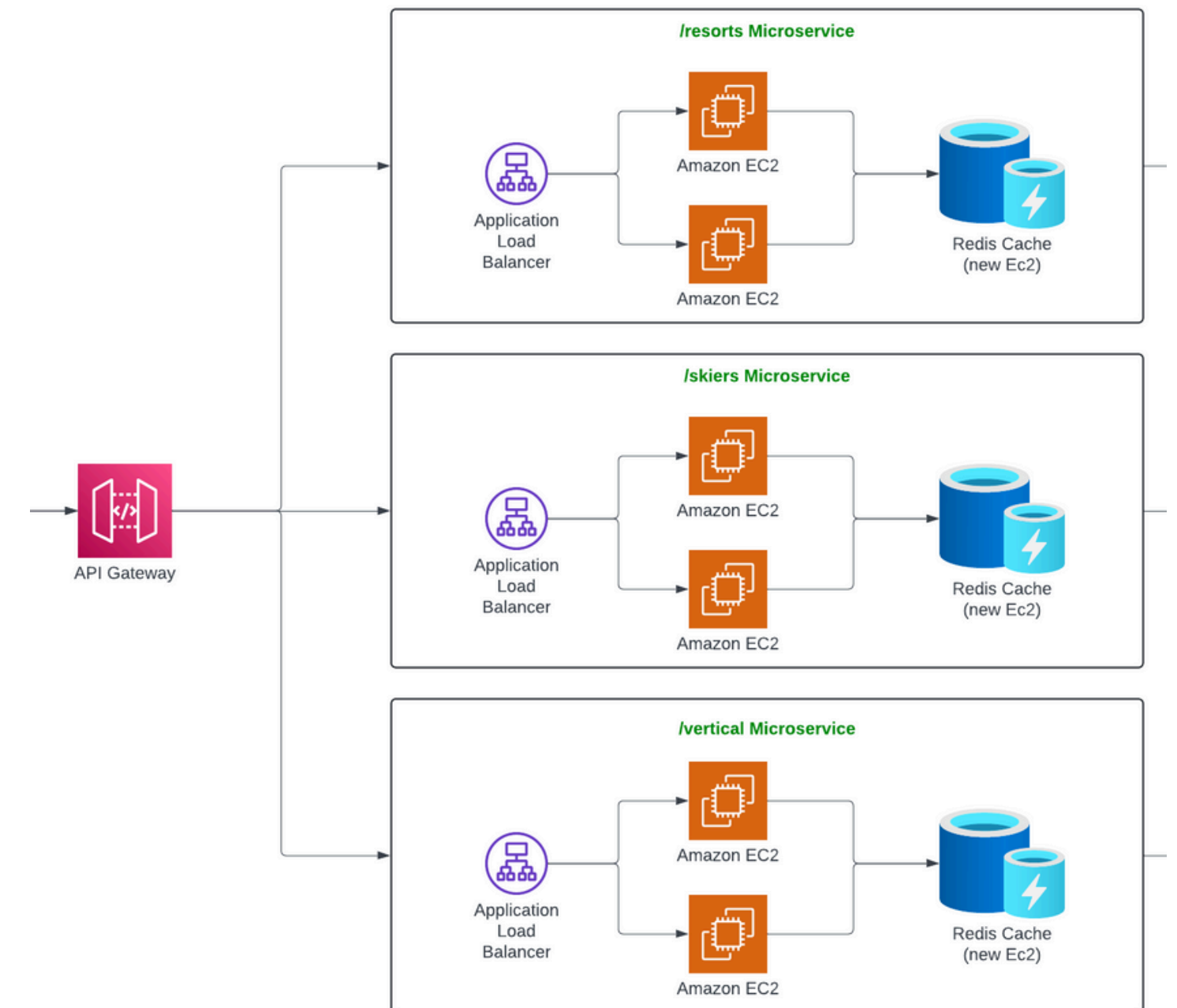


High Demand of one Service

Can also cause **cold starts** if one service is not as active as others

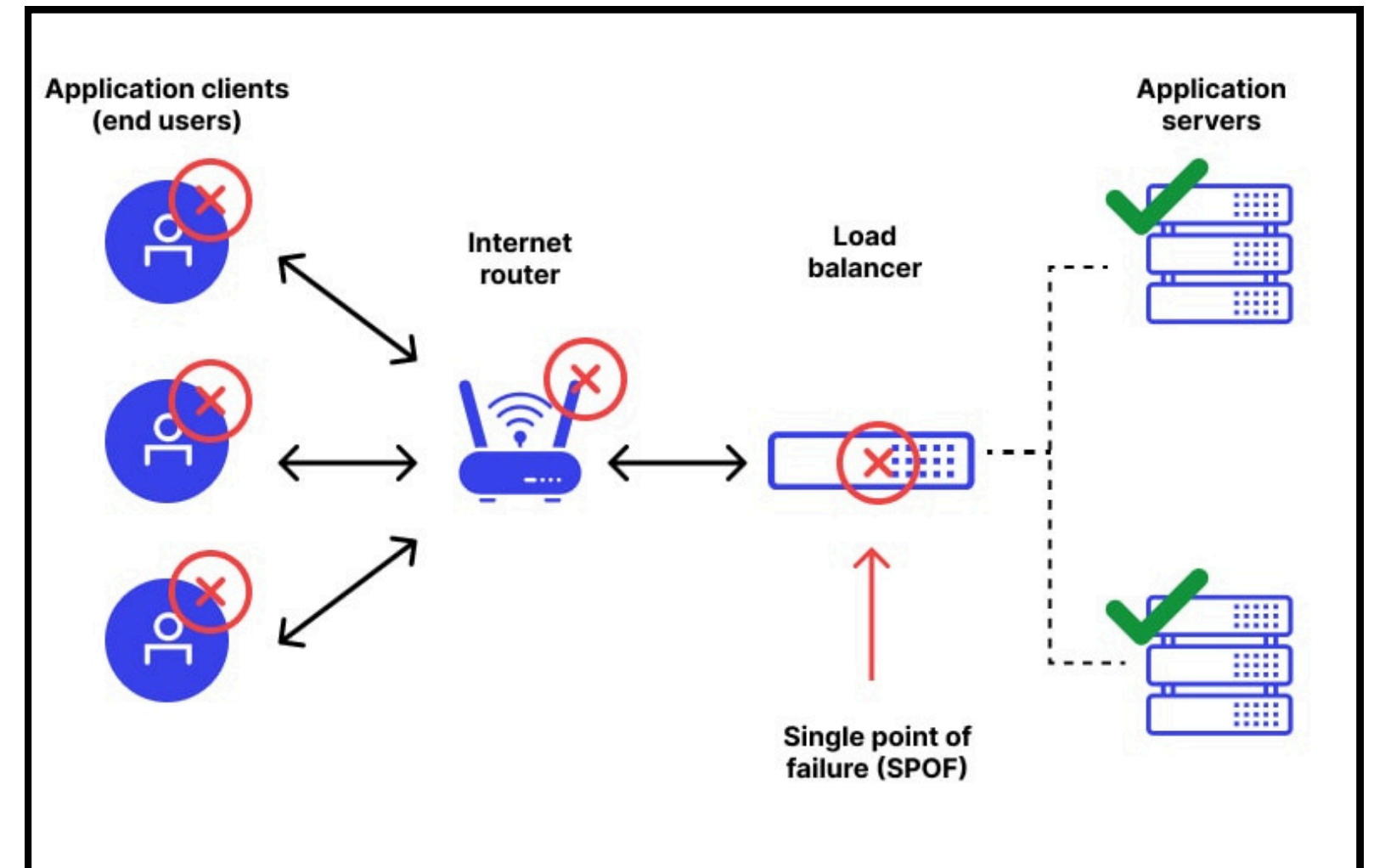
Monolith vs Microservice

- Microservices have **increased complexity** in **development** and **operations**. It is more difficult to **set up** and **manage** them.
- Code is also redundant, for example Utils, Database Connectors etc.
- Running **multiple microservice instances**, each with its **own cache**, **load balancers** etc can consume more resources compared to a **monolith**. This increases resource and network overhead.



API Gateway- SPOF

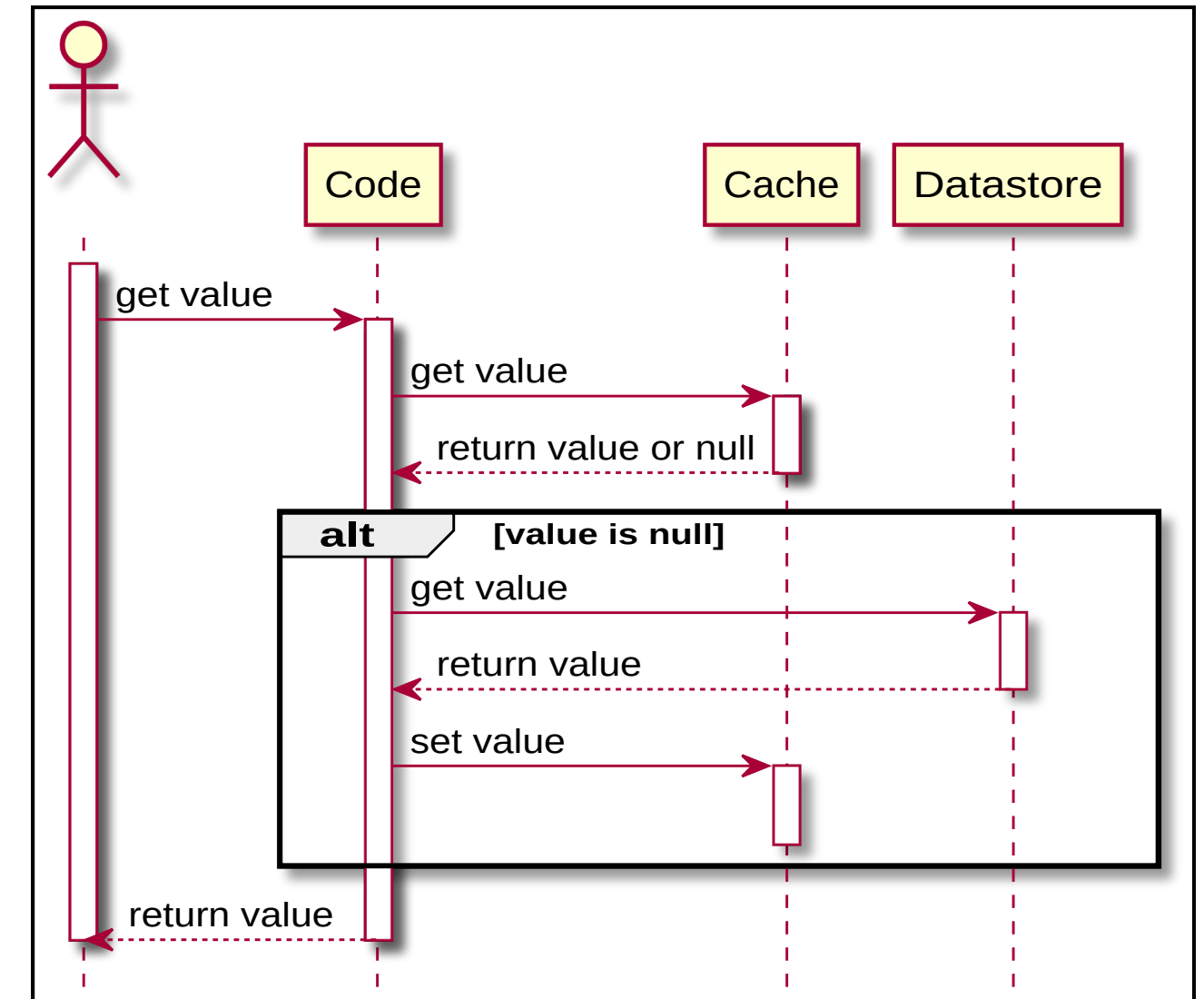
- Using API Gateway introduces a single point of failure. If the API Gateway **goes down**, **all incoming requests may be affected**, leading to service disruptions.
- Introducing an additional layer of abstraction can lead to increased **latency** and **overhead**, especially if the API Gateway performs extensive request processing or transformation.



Stale Cache



- A stale cache refers to a situation where the data stored in a cache is **outdated** or no longer reflects the most current state of the original data source.
- This can happen when the cached **data has expired** or the original data has been **updated** or changed, but the cache still holds the old information.
- Hence, systems **must** regularly **update** and **refresh** their caches to reflect the latest data.



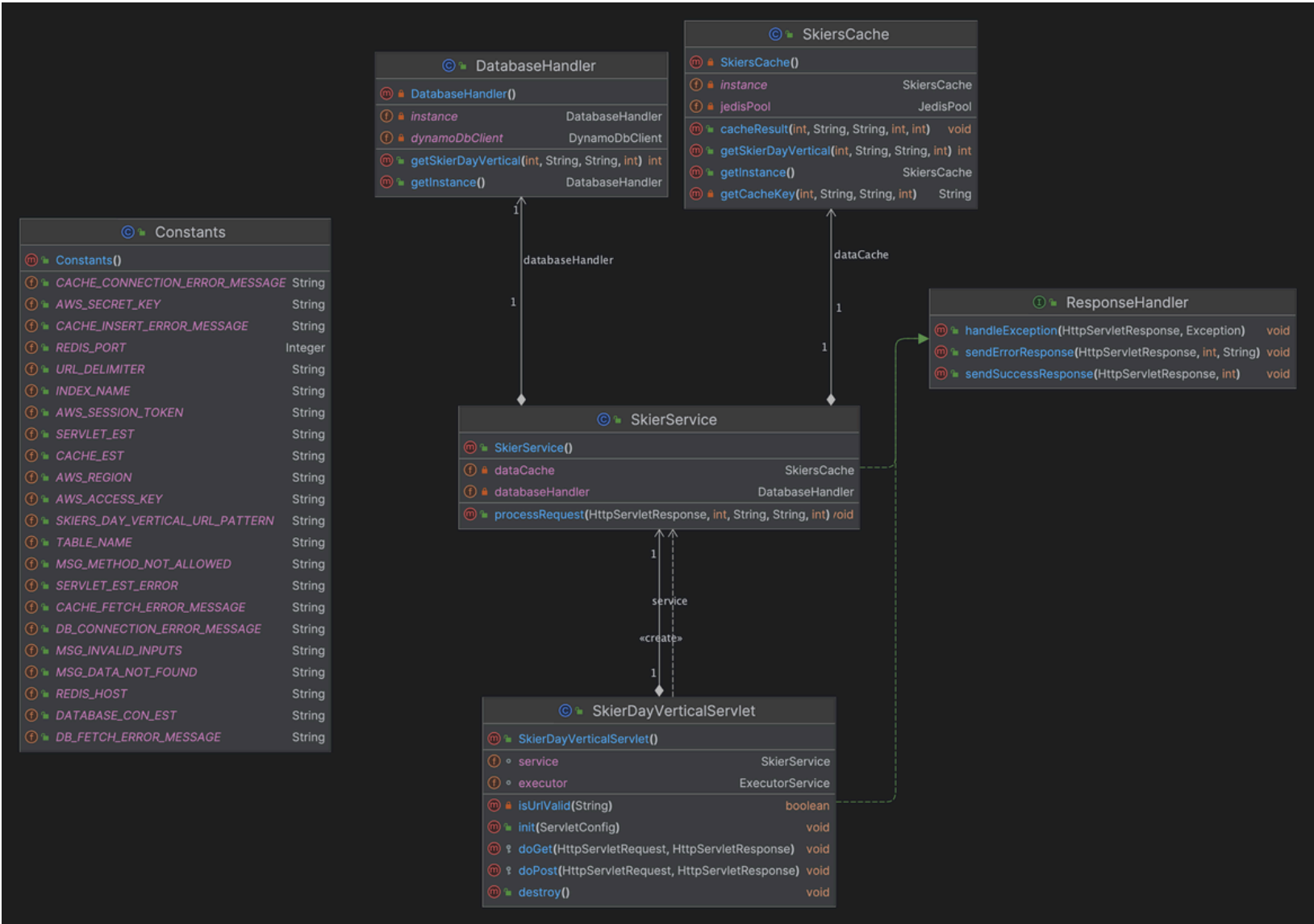
Availability **VS** Consistency

- Replicating data across **multiple regions** ensures that your application remains available even during a regional outage or failure.
- However, ensuring consistency across **replicas** in different regions can introduce replication **lag**.
- There might be a **delay** between writes in one region and the propagation of those writes to other regions, leading to **eventual consistency** rather than immediate consistency. Eventual consistency offers better performance but may result in temporarily stale data.
- Last Writer wins reconciliation.

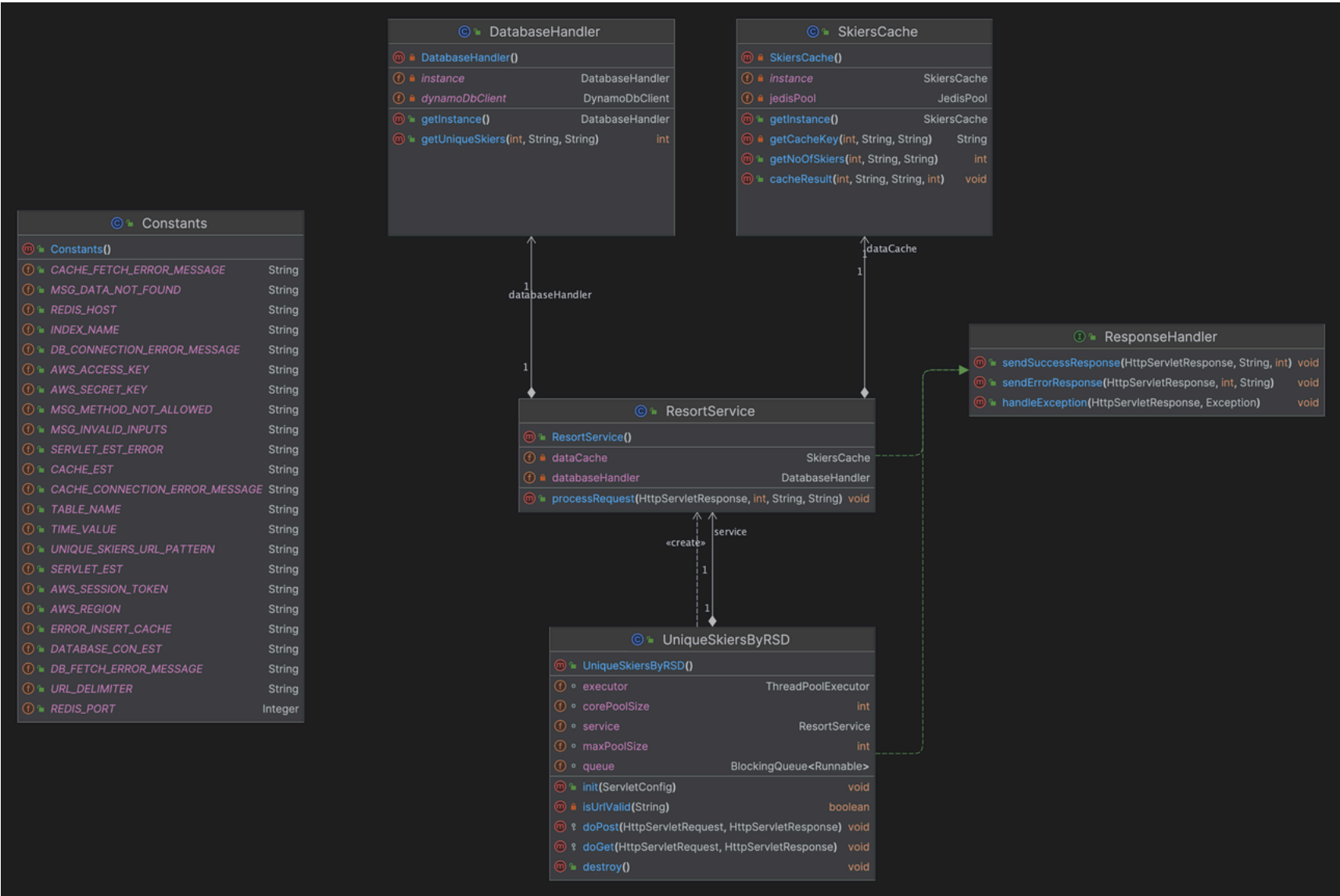


Class Diagram

/resorts

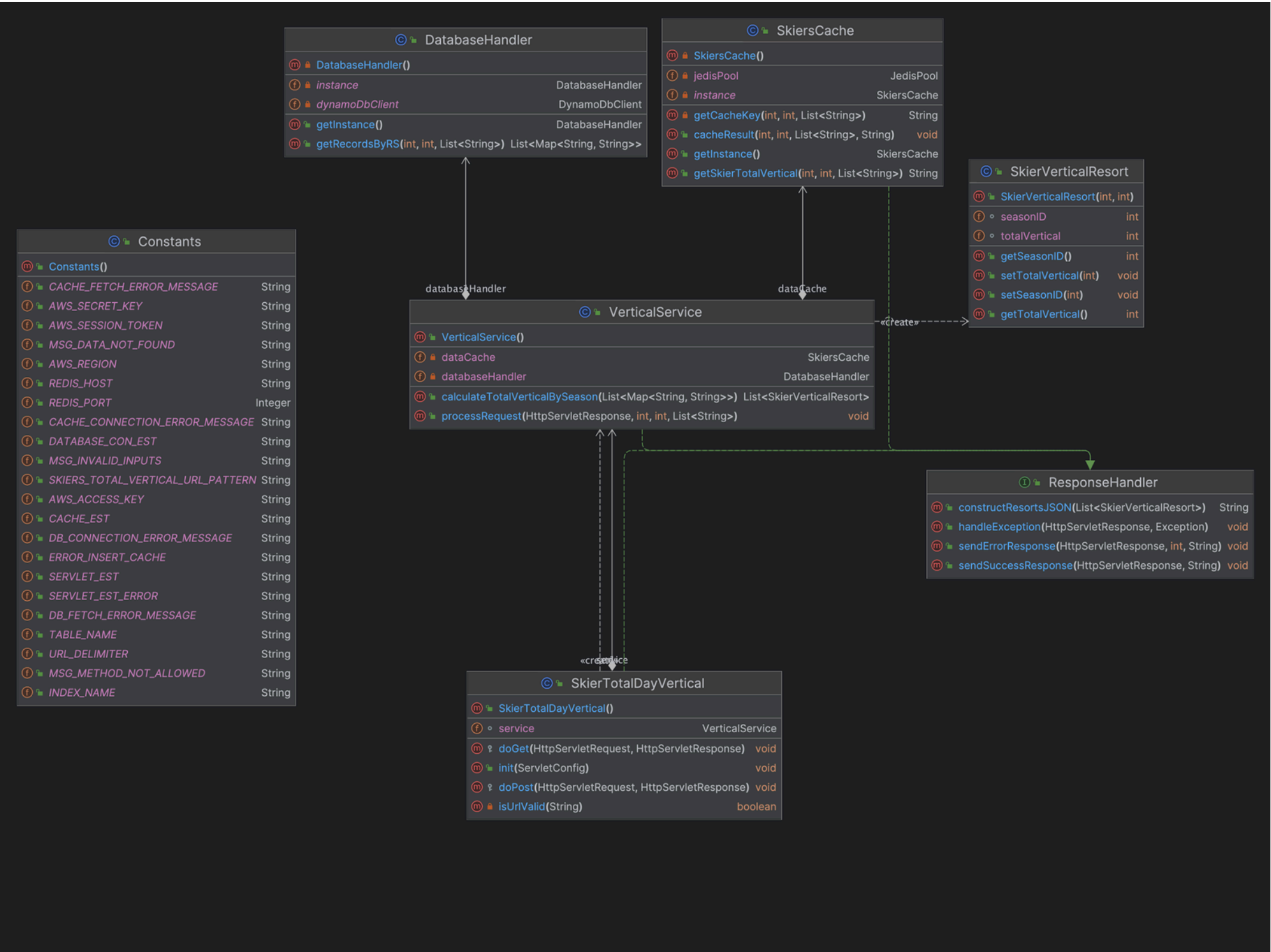
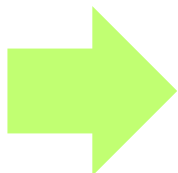


/skiers



Class Diagram

/skiers/{}/vertical



Data Model

☐	liftID (... ▼	time (Nu... ▼	dayID ▼	resortID ▼	seasonID ▼	skierID
☐	7	7	2	1	2024	29362
☐	7	20	1	1	2024	21618
☐	7	21	1	1	2024	98359
☐	7	30	1	1	2024	31067
☐	7	50	2	1	2024	44635
☐	7	112	2	1	2024	47949

Records with LiftID and Time as Keys



Using GSI fairly reduced the RCU units



SkierLiftRidesData_Final

Actions ▾

Explore table items

<

Overview

Indexes

Monitor

Global tables

Backups

Exports and streams

>

Global secondary indexes (2) [Info](#)

Delete

Create index

Find indexes

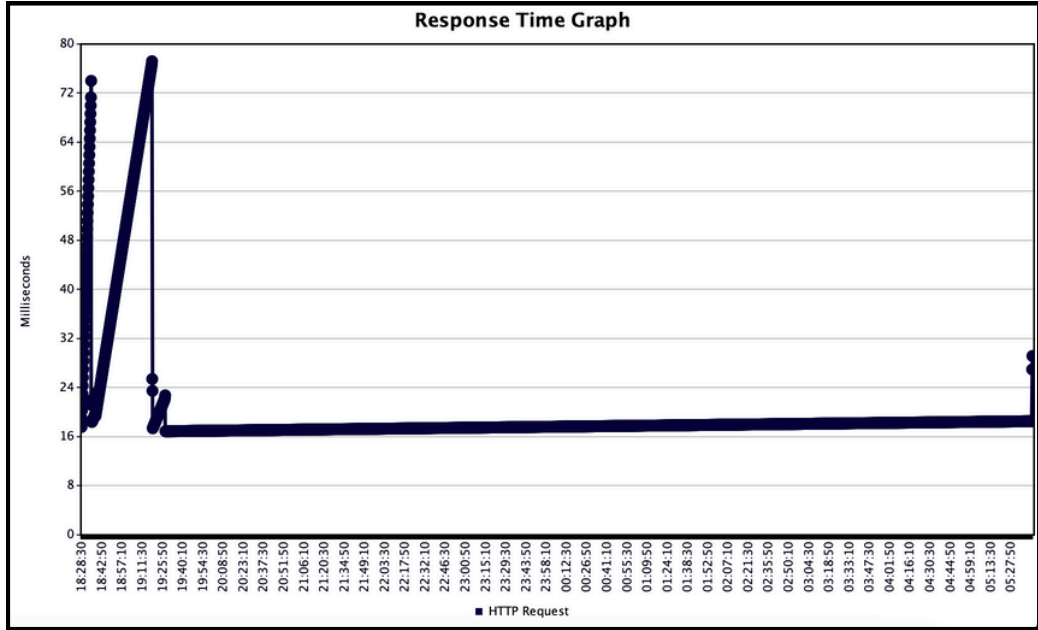
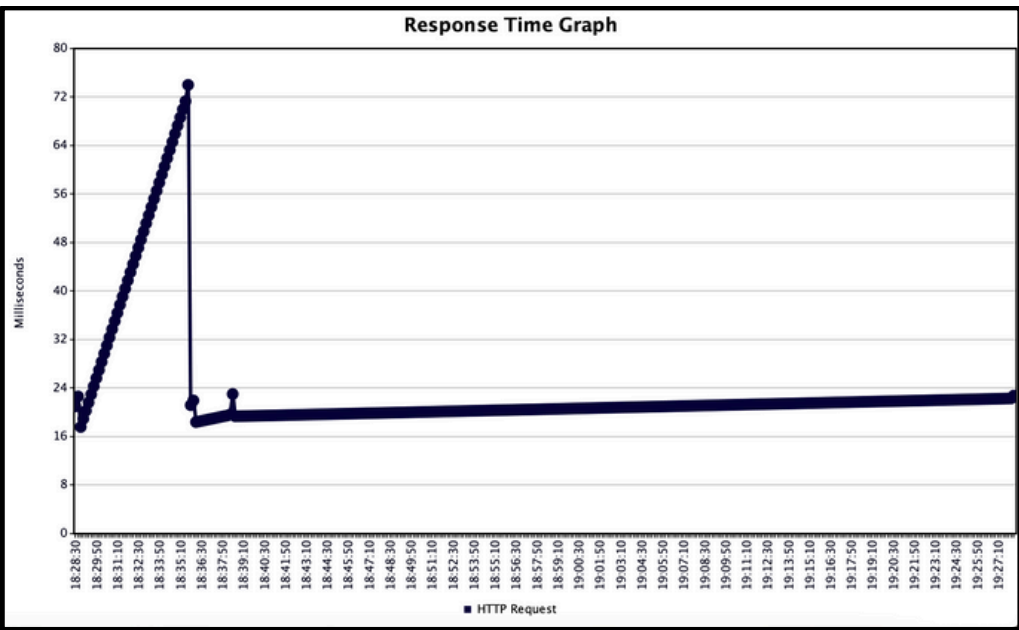
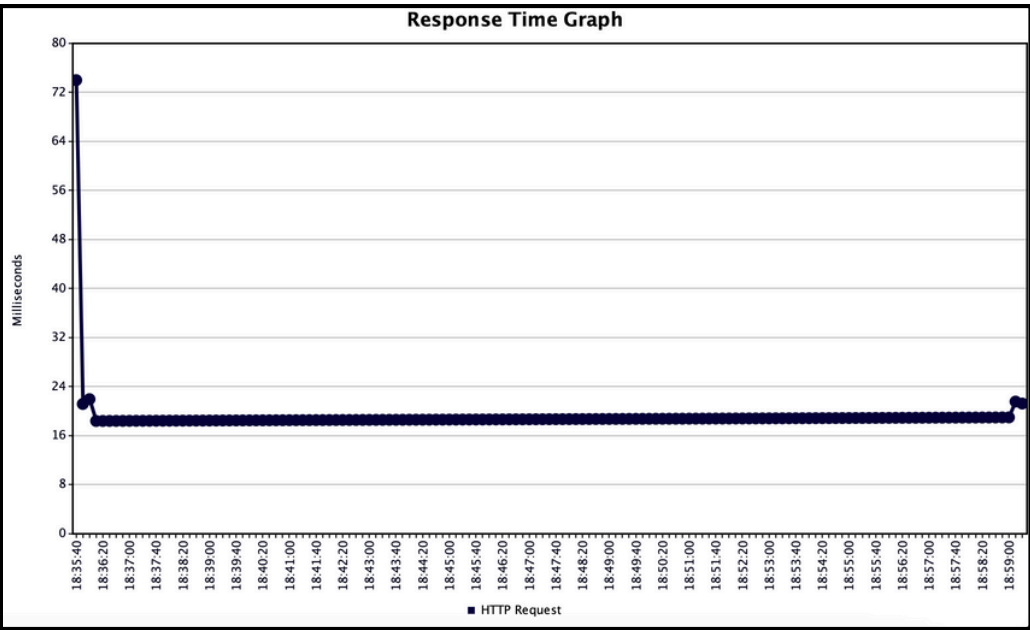
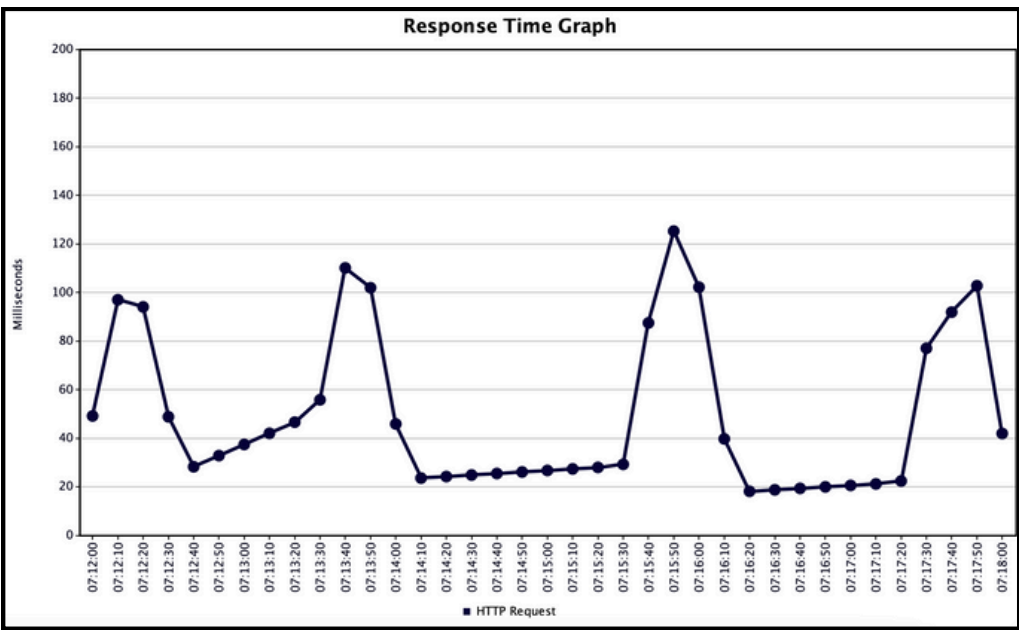
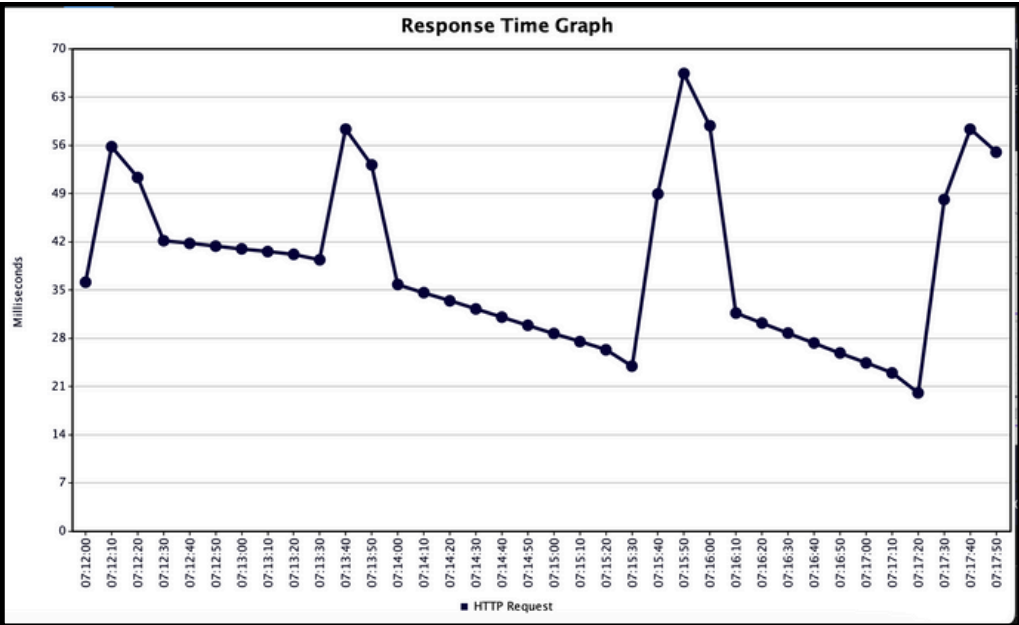
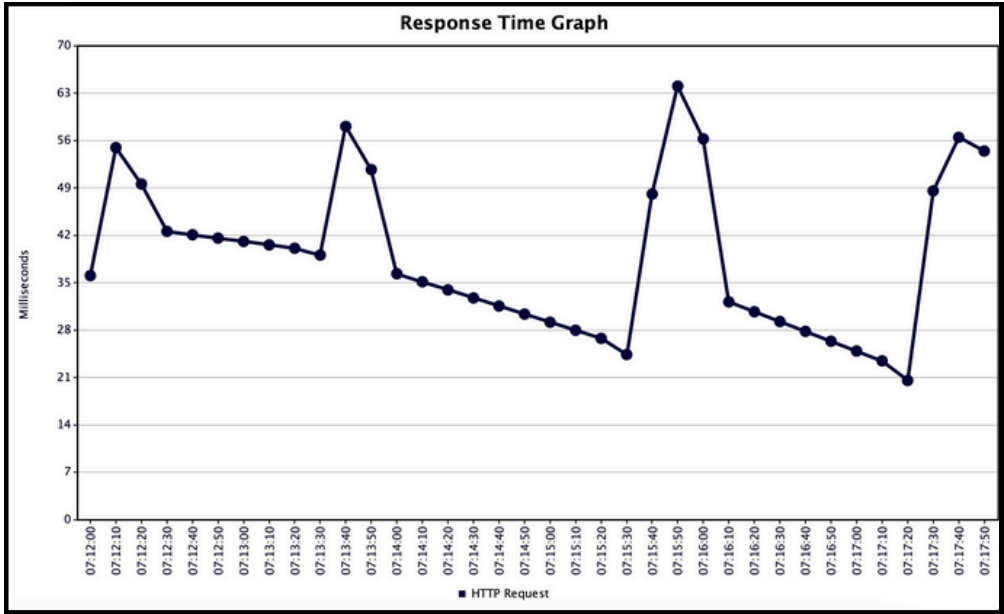
<

1

>

	Name ▲	Status ▾	Partition key ▾	Sort key ▾	Read capacity
☐	resortID-dayID-index	✔ Active	resortID (Number)	dayID (String)	5 Auto scaling is
☐	skierID-resortID-index	✔ Active	skierID (Number)	resortID (Number)	1 Auto scaling is

Response Time Graph for 3 Test Runs of 3 APIs



Test 1

[illegible][illegible][illegible]

Test 2

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
HTTP Requ...	64000	21	19	30	37	59	13	1051	0.00%	2970.3/sec	655.54	545.32
TOTAL	64000	21	19	30	37	59	13	1051	0.00%	2970.3/sec	655.54	545.32

Label	# Samples	Average ↑	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
HTTP Requ...	64000	22	19	35	41	58	12	239	0.00%	3048.6/sec	615.62	577.25
TOTAL	64000	22	19	35	41	58	12	239	0.00%	3048.6/sec	615.62	577.25

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Through...	Receive...	Sent KB...
HTTP R...	64000	27	22	45	54	87	12	492	0.00%	2616.2/...	94.73	500.48
TOTAL	64000	27	22	45	54	87	12	492	0.00%	2616.2/...	94.73	500.48

Thank you.